

Ethan Patterson

Algo HW3 Q1

a.) The algorithm used here is an implementation of “select”, which “selects” the Kth smallest value in a given array. It splits the values in the array into groups of 5, sorts the groups of 5 (constant time for each sort since constant size), and finds the median in each group. In the case of the size of the array not being multiples of 5, it grabs the larger of the two median values (ex. Size 4 has pos 2 and 3 as its “medians” and pos 3 is used). Then we grab all the medians, and recur the function to find the median of the medians. Our K for the recur is $n/5 * n/2$ (or $n/10$). Once we have the median of medians, we use it as a pivot to split the main array into “less than or equal to the median” and “greater than the median”. Then if K is definitely somewhere in the “left side” ($\leq$ median, check the size of the left side against K), we recur with the left side of the array and k again. If k is right on our split, we return our median of median as the Kth smallest element. If k is somewhere in the “right side” (check the size of the right side against K), we recur with the right side of the array and $K - \text{left side.size}$ (as those are all smaller than K so the Kth smallest needs to be adjusted). Once this breaks it down to either having $K < 5$ or $K = \text{median of medians}$, we either brute force it for the former, or simply exit early and return for the latter.

b.) Pseudocode:

Main function:

Number of coordinates “n” is input

Coordinates of police “coords_x” and “coords_y” is input

$x_med = \text{select-median-of-medians}(\text{coords_x}, n / 2)$

$y_med = \text{select-median-of-medians}(\text{coords_y}, n / 2)$

for all police coordinates:

sum += absolute value ($x_med - \text{coord } x$) + absolute value ($y_med - \text{coord } y$)

sum is output

Select-median-of-medians (array, k):

If k is less than 5:

Sort the array

Return array[k]

Number of groups “num_groups” is the size of array divided by 5

“grouped_coords” is a 2d array, of size num_groups, with each inner array of size 5.

Insert each element into the correct group and within each group

“M” is an array of size num_groups

For each group:

Sort the group

$M[\text{some index}] = \text{middle of the group}$

"Median" is the middle element of M, or median of M

"L" is the array of all elements "x" in "array" such that $x \leq \text{Median}$

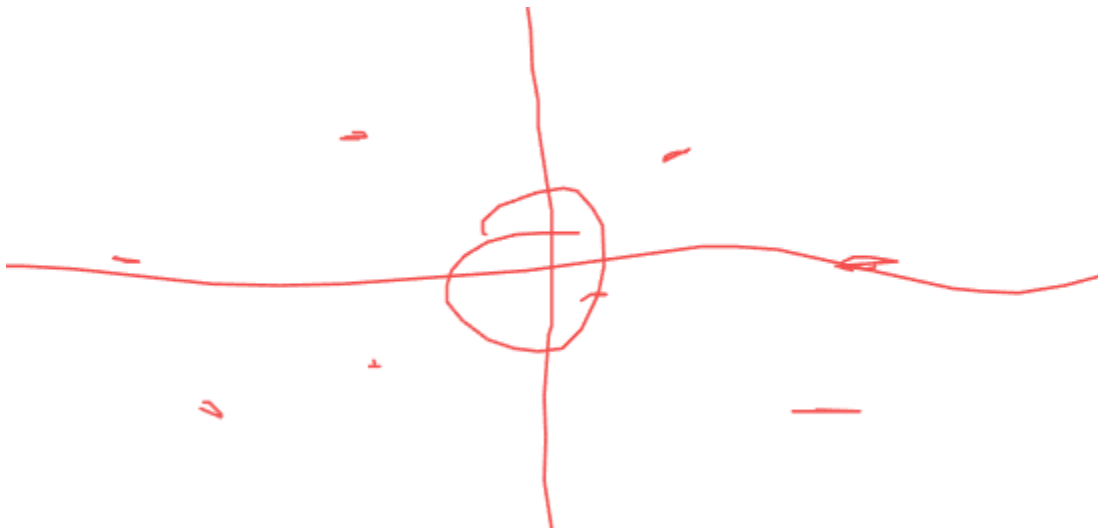
"R" is the array of all elements "x" in "array" such that $x > \text{Median}$

If k is less than length of L, return select-median-of-medians(L, k)

If k is equal to length of L, return Median

If k is greater than length of L, return select-median-of-medians(R, k-length of L)

c.) Proof of correctness:



Say this is our x and y grid presented in the problem. The dots are our police coordinates. The shortest distance for them to travel in both the x and y is marked by the line. The circled intersection is our "shortest travel distance location". This works because we can use the select algorithm to find the median of our x values (the vertical line), searching for the $n/2$ smallest value (right in the middle, thus the median). Then we can use it again to find the median for the y values in the same way, generating our coordinate pair for the intersection. After that all we need to do is sum up all the travel distances using $\sum_{i=1}^n |x_{best} - x_i| + |y_{best} - y_i|$ to generate the minimum possible sum of distances to every police car.

d.) $O(n)$

e.) We are using select with median of medians as pivot. We only have to do one sub problem and they are guaranteed to be $7n/10$ at most, which is smaller. As such, our runtime is $O(n)$ worst case with the select algorithm, and all code and logic implementations inside the select exclusively use linear or constant time pieces in order to maintain this.